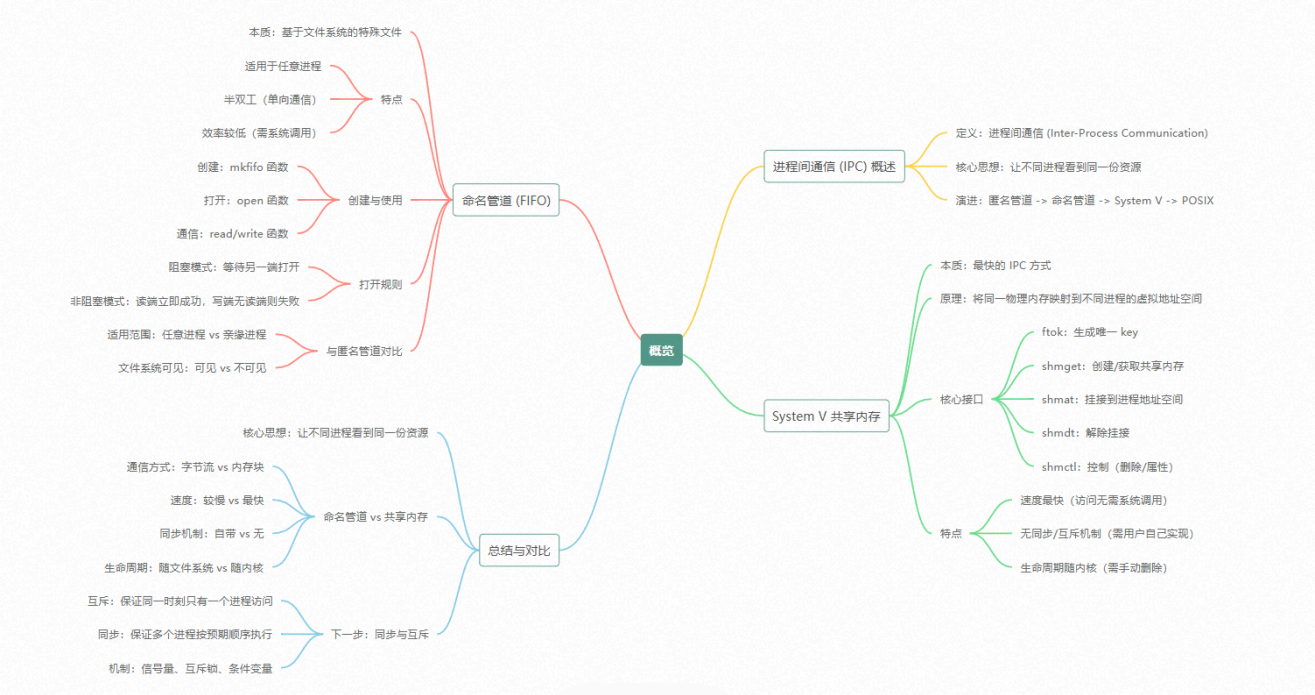
2026-7-30 Linux进程--命名管道和共享内存

- 1. 命名管道（FIFO）
  - 1.1 从匿名管道到命名管道
  - 1.2 命名管道为什么是一个“文件”
  - 1.3 创建命名管道
  - 1.4 匿名管道与命名管道
  - 1.5 命名管道的打开规则
  - 1.6 Server 与 Client 通信示例
- 2. System V 共享内存
  - 2.1 为什么共享内存最快
  - 2.2 共享内存的组织方式
  - 2.3 key、ftok 与 shmid
  - 2.4 共享内存核心接口
    - 2.4.1 shmget：创建或获取共享内存
    - 2.4.2 shmat：挂接到进程地址空间
    - 2.4.3 shmdt：解除当前进程的挂接
    - 2.4.4 shmctl：查询属性与删除资源
  - 2.5 生命周期、查看与清理
  - 2.6 容量、权限与越界风险
  - 2.7 共享内存访问示例
  - 2.8 共享内存为什么必须考虑同步
- 3. 对比与结论
  - 3.1 命名管道与共享内存的核心差异
  - 3.2 最终理解

1. 命名管道（FIFO）



1.1 从匿名管道到命名管道

匿名管道能让父子进程通信，但它依赖 `fork()` 继承的文件描述符。两个没有共同祖先、互不相干的进程，没有共享的文件描述符表，因此不能直接通过匿名管道建立通信。

命名管道（FIFO，First In First Out）保留了管道的内核缓冲区和读写语义，同时利用文件系统中的路径名给通信对象增加一个双方都能找到的名字：

```
进程 A -- 路径名 --> FIFO 文件 --> 路径名 <-- 进程 B
```

因此，命名管道解决的是匿名管道“没有名字、只能依赖继承关系”的限制。它仍然遵循同一个 IPC 核心思想：

> **让不同的进程看到同一份资源。**

1.2 命名管道为什么是一个“文件”

```
ZL@VM-0-14-ubuntu:~/code/lesson23$ ll -li
total 8
790173 drwxrwxr-x  2 ZL ZL 4096 Jul 30 21:22 ./
655390 drwxrwxr-x 27 ZL ZL 4096 Jul 30 21:22 ../
790181 prw-rw-r--  1 ZL ZL    0 Jul 30 21:22 named_pipe|
```

命名管道在文件系统中表现为特殊类型的文件：

1. 使用 `ls -l` 查看时，类型位为 `p`。
2. 目录项中可以存在文件名，因此任意进程都能通过同一个路径访问它。
3. 文件中不保存普通磁盘数据，大小长期显示为 `0`。
4. 实际通信数据存放在内核管道缓冲区，读写后才被消费。

路径名和文件名最终定位同一个 inode。两个进程只要打开同一路径所对应的同一个 FIFO，就会连接到同一个通信对象。文件相关的核心数据结构和缓冲区只存在一份，不需要为每个打开者复制一份完整文件。



例如，一个终端执行：

```
echo "hello" > named_pipe
```

另一个终端执行：

```
cat < named_pipe
```

`echo` 和 `cat` 是两个独立进程，但通过同一个命名管道完成了数据传输。

创建同一个通信对象时应满足“一方创建、另一方获取并打开”。若双方分别创建不同路径的 FIFO，它们得到的是不同资源，不能互相通信。

1.3 创建命名管道

命令行可以直接创建：

```
mkfifo named_pipe
ls -l named_pipe
```

程序中使用：

```
#include <sys/stat.h>
#include <sys/types.h>
```

```
int mkfifo(const char *pathname, mode_t mode);
```

- 成功返回 `0`。
- 失败返回 `-1`，并设置 `errno`。
- `EEXIST`：路径对应的文件已经存在。
- `EACCES`：路径权限不足。
- `ENOENT`：父目录不存在。

常见权限参数是 `0666`，但实际权限还会被进程的 `umask` 削减。需要固定权限时可先调整 `umask`：

```
#include <sys/stat.h>
#include <cerrno>
#include <cstring>
#include <iostream>
#include <unistd.h>

int main()
{
    umask(0);
    if (mkfifo("./named_pipe", 0666) == -1 && errno != EEXIST)
    {
        std::cerr << "mkfifo: " << std::strerror(errno) << '\n';
        return 1;
    }
    return 0;
}
```

创建后使用普通的 `open()` 打开，而不是 `fopen()`。通信阶段继续使用 `read()` 和 `write()`。

```
ZL@VM-0-14-ubuntu:~/code/lesson23$ ll
total 60
drwxrwxr-x  2 ZL ZL  4096 Jul 31 12:18 ./
drwxrwxr-x 27 ZL ZL  4096 Jul 30 21:51 ../
-rwxrwxr-x  1 ZL ZL 16328 Jul 31 12:18 Client*
-rw-rw-r--  1 ZL ZL    68 Jul 30 21:58 Client.cc
p---------  1 ZL ZL     0 Jul 31 12:17 fifo|
-rw-rw-r--  1 ZL ZL   153 Jul 30 22:07 Makefile
prw-rw-r--  1 ZL ZL     0 Jul 30 21:39 named_pipe|
-rw-rw-r--  1 ZL ZL   896 Jul 31 12:18 Pipe.hpp
-rwxrwxr-x  1 ZL ZL 17880 Jul 31 12:18 Server*
-rw-rw-r--  1 ZL ZL    90 Jul 31 12:09 Server.cc
ZL@VM-0-14-ubuntu:~/code/lesson23$ ./Server
mkfifo error: File existserrno: 17
```

如果权限掩码处理不当，会出现创建结果不符合预期或后续打不开的问题。排查时应同时检查：

- `mkfifo()` 是否成功。
- `errno` 是否为 `EEXIST`。
- 文件类型是否为 `p`。
- `umask` 与实际权限是否满足读写需求。

1.4 匿名管道与命名管道

对比项	匿名管道	命名管道（FIFO）
创建方式	`pipe()`	`mkfifo()`
打开方式	`pipe()` 同时返回读端和写端	使用 `open()` 打开路径
是否有文件名	没有	有
是否能被无关进程访问	通常不能，依赖继承	可以
底层通信语义	字节流	字节流
通信方向	单向	单向
生命周期	所有端点关闭后由内核回收	文件系统中的名字独立存在，通信对象随打开引用管理

二者最关键的差别在创建和打开方式。FIFO 一旦打开，`read()`、`write()`、阻塞、EOF 和 `SIGPIPE` 等行为都与匿名管道基本一致。

1.5 命名管道的打开规则

打开方式	`O_NONBLOCK` 状态	行为
`O_RDONLY`	未设置	阻塞，直到有进程以写方式打开同一 FIFO
`O_RDONLY`	已设置	立即返回可用的 fd
`O_WRONLY`	未设置	阻塞，直到有进程以读方式打开同一 FIFO
`O_WRONLY`	已设置	若当前没有读端，立即失败，`errno = ENXIO`

```
// 非阻塞读端
int rfd = open("./named_pipe", O_RDONLY | O_NONBLOCK);

// 非阻塞写端：没有读端时会失败
int wfd = open("./named_pipe", O_WRONLY | O_NONBLOCK);
```

非阻塞读端可以先成功打开并等待后续写入；非阻塞写端若没有接收者则没有意义，因此内核直接返回 `ENXIO`。

打开成功以后，仍需处理这些边界情况：

- 管道为空时，阻塞读取会等待数据。
- 所有写端关闭后，读端在取完剩余数据后得到 `read() == 0`。
- 所有读端关闭后，继续写入会触发 `SIGPIPE`；忽略该信号时可检查 `errno == EPIPE`。
- FIFO 仍然是字节流，不保存消息边界。

1.6 Server 与 Client 通信示例

服务端负责创建 FIFO、打开读端并等待消息：

```
#include <cerrno>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <iostream>
#include <sys/stat.h>
#include <unistd.h>
#include <fcntl.h>

int main()
```

```
{
    const char *path = "./named_pipe";
    umask(0);

    if (mkfifo(path, 0666) == -1 && errno != EEXIST)
    {
        std::perror("mkfifo");
        return 1;
    }

    int fd = open(path, O_RDONLY); // 阻塞到写端打开
    if (fd == -1)
    {
        std::perror("open");
        return 1;
    }

    char buffer[1024];
    ssize_t n = read(fd, buffer, sizeof(buffer) - 1);
    if (n > 0)
    {
        buffer[n] = '\0';
        std::cout << "server received: " << buffer << '\n';
    }

    close(fd);
    unlink(path);
    return 0;
}
```

客户端打开写端并发送数据：

```
#include <cstdio>
#include <cstring>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd = open("./named_pipe", O_WRONLY); // 阻塞到读端出现
    if (fd == -1)
    {
        std::perror("open");
        return 1;
    }
```

```
    const char *message = "hello from client";
    write(fd, message, std::strlen(message));
    close(fd);
    return 0;
}
```

运行顺序通常是先启动服务端，再启动客户端。即使服务端先执行到 `open()`，它也会阻塞等待客户端的写端；删除 FIFO 使用 `unlink()`，删除目录项不会影响已经打开但尚未关闭的端点。

2. System V 共享内存

ⓘ 课堂板书：从 FIFO 过渡到共享内存 ›

命名管道

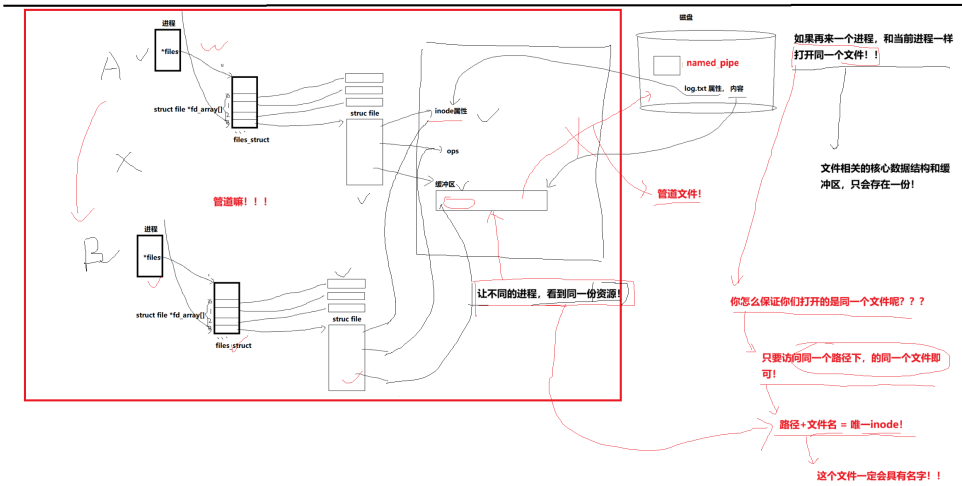
让不同的进程，看到同一份资源！

如果两个进程，毫无关系，那么如何通信呢？？

- 只能用于具有共同祖先的进程（具有亲缘关系的进程）之间进行通信；通常，一个管道由一个进程创建，然后该进程调用fork，此后父、子进程之间就可应用该管道。
- 管道提供流式服务
- 一般而言，进程退出，管道释放，所以管道的生命周期随进程
- 一般而言，内核会对管道操作进行同步与互斥
- 管道是半双工的，数据只能向一个方向流动；需要双方通信时，需要建立起两个管道

3-8 验证管道通信的4种情况

- 读正常&&写满
- 写正常&&读空
- 写关闭&&读正常
- 读关闭&&写正常

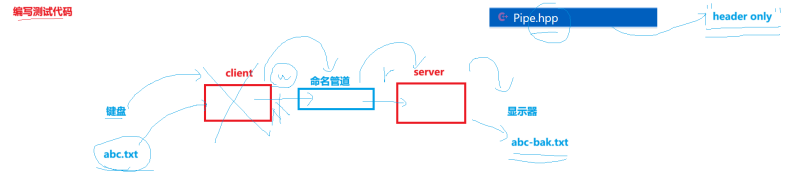



```
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson36$ mkfifo named_pipe
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson36$ ll
total 8
drwxrwxr-x  2 whb whb 4096 Nov 17 19:09 ./
drwxrwxr-x 31 whb whb 4096 Nov 17 19:08 ../
prw-rw-r--  1 whb whb    0 Nov 17 19:09 named_pipe|
```

```
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson36$ ll -li
total 8
655656 drwxrwxr-x  2 whb whb 4096 Nov 17 19:09 ./
135544 drwxrwxr-x 31 whb whb 4096 Nov 17 19:08 ../
655671 prw-rw-r--  1 whb whb    0 Nov 17 19:09 named_pipe|
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson36$ pwd
/home/whb/code/116_117/code/lesson36
```

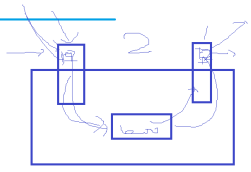
```
prw-rw-r--  1 whb whb    0 Nov 17 19:09 named_pipe|
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson36$ echo "aaaaa"
aaaaa
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson36$ echo "aaaaa" > name
d_pipe
```

```
prw-rw-r--  1 whb whb    0 Nov 17 19:09 named_pipe|
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson36$ cat < named_pipe
aaaaa
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson36$
```

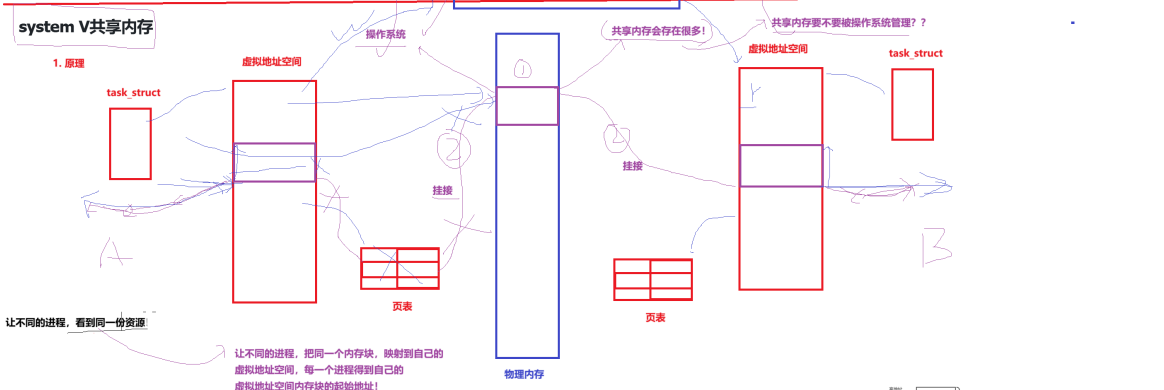


我们把进程间通信，我们叫做进程IPC Inter-Process Communication

1. 匿名 && 命名管道 --- 基于文件 -- 基于已有的文件内核代码，进行的改良本质是在复用代码
2. System V进程间通信 —— 本主机通信
   - 共享内存
   - 消息队列
   - 信号量

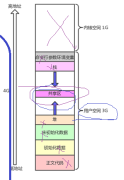


system V共享内存



2. 理解
   1. 映射到进程的虚拟地址空间，共享区
   2. 共享内存原理，是一个简化版本的动态库映射！
   3. 共享内存管理结构体 + 共享内存本身 = 共享内存
   4. 使用共享内存的步骤：1. 创建 2. 关联挂接 3. 使用 () 4. 去关联 5. 释放共享内存

共享内存特点：
1. 访问共享内存，不需要系统调用，因为shm已经映射到了进程的用户共享区了！
2. 写端数据拷贝的shm，其他端立马能看到！！
   - 共享内存，是所有进程间通信方式中，速度最快的！！
   - 拷贝次数少，直接映射，不需要系统调用！
3. 缺点：没有资源的保护机制，没有同步或者互斥！
   - sem由用户自己完成保护！



3. 共享内存是什么？为什么？怎么用？

   采用多个进城，使用虚拟地址空间映射的方式，让不同的进程看到同一个内存块 --- 共享内存

4. 认识接口：

NAME

shmget - allocates a System V shared memory segment

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

大小，必须是4096的整数倍！

命名管道：
1. 看到同一个资源
2. 一方创建，一个进程获取(使用)

获取shm

我总要获取一个共享内存

#include <sys/shm.h>

int shmget(key_t key, size_t size, int shmflg);

IPC_CREAT and IPC_EXCL

创建共享内存的

IPC_EXCL: 不能单独使用

IPC_CREAT：可以单独传递，如果创建的共享内存不存在，就创建之，如果存在，我就获取它

IPC_CREAT | IPC_EXCL: 如果创建的共享内存不存在，就创建之，如果存在，出错返回！

共享内存的key值，在内核中，标识shm的唯一性！

需要用户来传递吗？？

为什么？怎么办到的？？

我们怎么知道共享内存存在还是不存在呢？？

我只要新的共享内存！

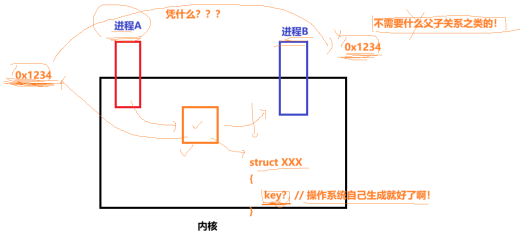
为什么不由内核来做？？

做不到！！！！

共享内存未来要有自己的唯一值！！

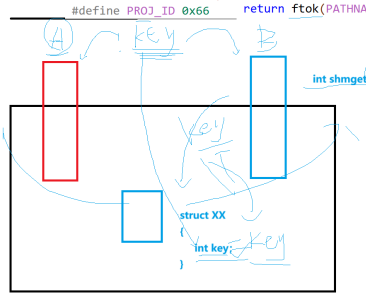
RETURN VALUE

On success, a valid shared memory identifier is returned. On error, -1 is returned, and errno is set to indicate the error.



#define PATHNAME "/tmp"

#define PROJ_ID 0x66 return ftok(PATHNAME, PROJ_ID);



AB是如何约定的：程序员编码的时候，就提前写好了！！

int shmget(0x1234 ...)

NAME

ftok - convert a pathname and a project identifier to a System V IPC key

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

#include <sys/ipc.h>

key_t ftok(const char *pathname, int proj_id);

```
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson37$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x6602f807 0          whb        0          4096       0
```

查看系统中的已经有的共享内存资源

细节1：共享内存(包括：system IPC)，它的生命周期随内核！

用户不主动删除ipc资源，ipc资源会和你的OS一样，一直存在除非你重启系统！

细节2：

指令

代码

```
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson37$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x6602f807 0          whb        0          4096       0

whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson37$ ipcrm -m 0x6602f807
ipcrm: failed to parse argument: '0x6602f807'
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson37$ ipcrm -m 0
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson37$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
```

fd 和 inode number

key vs shmid:

1. key只在内核中，标识共享内存的唯一性！用户使用共享内存，不用这个key！

2. shmid 只在用户中使用，你自己代码中，使用shmid来访问共享内存

删除共享内存：ipcrm -m 不能跟key，跟的是shmid?

指令的执行，本质也是用户代码在执行！！！

```
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson37$ ./Server
key=0x6602f807, _shmid = 1
whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson37$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x6602f807 1          whb        0          4096       0

whb@iv-ye4ege8iyo5i3z3clix9:~/code/116_117/code/lesson37$ ./Server
shmget: File exists
```

为什么？

key冲突了？？

还是shmid？？

NAME

shmctl - System V shared memory control

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

#include <sys/shm.h>

int shmctl(int shmid, int op, struct shmid_ds *buf);

IPC_RMID

Mark the segment to be destroyed.

删除共享内存

缓冲区

对于共享内存

1. 删除是控制的方式之一

2. 也可以获取或者设置共享内存的属性，也是op 共享内存的方式

NAME

shmat, shmdt - System V shared memory operations

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

#include <sys/shm.h>

void *shmat(int shmid, const void *_Nullable shmaddr, int shmflg);

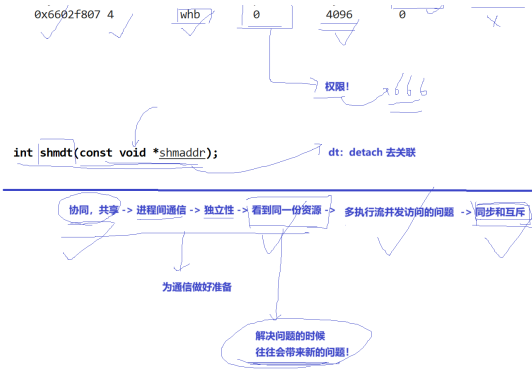
void *malloc(size_t size);

attach 附加，连接

挂接数

表达的是，当前有多少个进程把我attach到他的地址空间了！

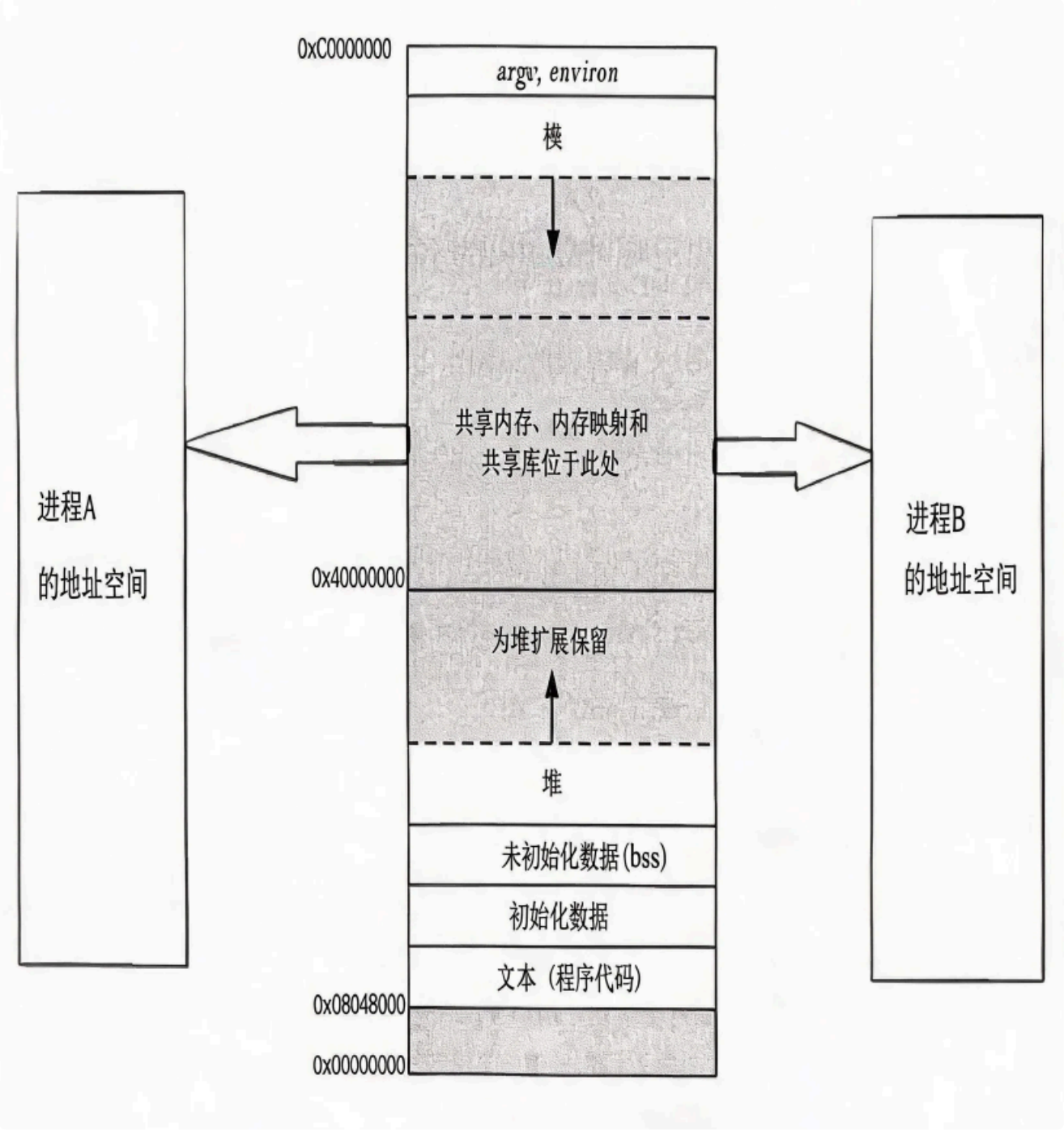
```
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
```


0x6602f807 4
whb
0
4096
0
权限!
666
int shmdt(const void *shmaddr);
dt: detach 去关联

协同，共享 -> 进程间通信 -> 独立性 -> 看到同一份资源 -> 多执行流并发访问的问题 -> 同步和互斥
为通信做好准备
解决问题的时候
往往会带来新的问题!

2.1 为什么共享内存最快

管道的每一次数据传输都要经过系统调用，并由内核缓冲区负责搬运。共享内存则先由内核创建一段共享物理内存，再将它映射到多个进程各自的虚拟地址空间。



建立映射后，进程访问共享内存就像访问自己的普通用户空间：

```
进程 A 的虚拟地址 --映射--> 同一段物理内存 <--映射-- 进程 B 的虚拟地址
```

两个进程通常得到不同的虚拟起始地址，但底层页面对应同一份物理内存。写入一个页面后，另一个进程能立即通过自己的映射看到变化。

共享内存速度快的主要原因是：

1. 映射完成后，传输数据不需要反复调用 `read()`、`write()`。
2. 数据不需要先从用户空间复制到内核缓冲区，再复制给另一个进程。
3. 进程直接访问映射页面，复制次数最少。

2.2 共享内存的组织方式

共享内存不是用户进程自己申请的普通堆内存，而是由操作系统统一创建和管理的资源。

系统中可能同时存在很多共享内存对象，因此内核采用“先描述、再组织”的方法：

```
管理结构体 + 实际内存页 = 一段共享内存
```

`shmid_ds` 一类结构保存共享内存的标识、权限、大小、创建者、挂接数以及访问时间等信息，内核再把这些描述结构组织起来统一管理。

共享内存的基本使用流程是：

1. 生成或约定 `key`。
2. 创建或获取共享内存段。
3. 将共享内存挂接到当前进程的虚拟地址空间。
4. 直接读写映射区域。
5. 解除挂接。
6. 不再需要时删除共享内存资源。

可以把共享内存理解为一个简化版的动态库或文件映射：内核负责建立页表映射，用户代码负责访问映射后的地址。

2.3 `key`、`ftok` 与 `shmid`

通信双方必须先约定如何找到同一个共享内存对象。`ftok()` 根据文件信息和项目编号生成 System V IPC 的 `key`：

```c
#include <sys/ipc.h>

key_t ftok(const char *pathname, int proj_id);
```

```c
key_t key = ftok("/tmp", 0x66);
if (key == -1)
{
    perror("ftok");
    return 1;
}
```

只要双方使用同一个仍然存在的文件路径和同一个 `proj_id`，通常就能得到相同的 key。路径指向的文件被删除或重建后，inode 可能变化，生成的 key 也可能变化。

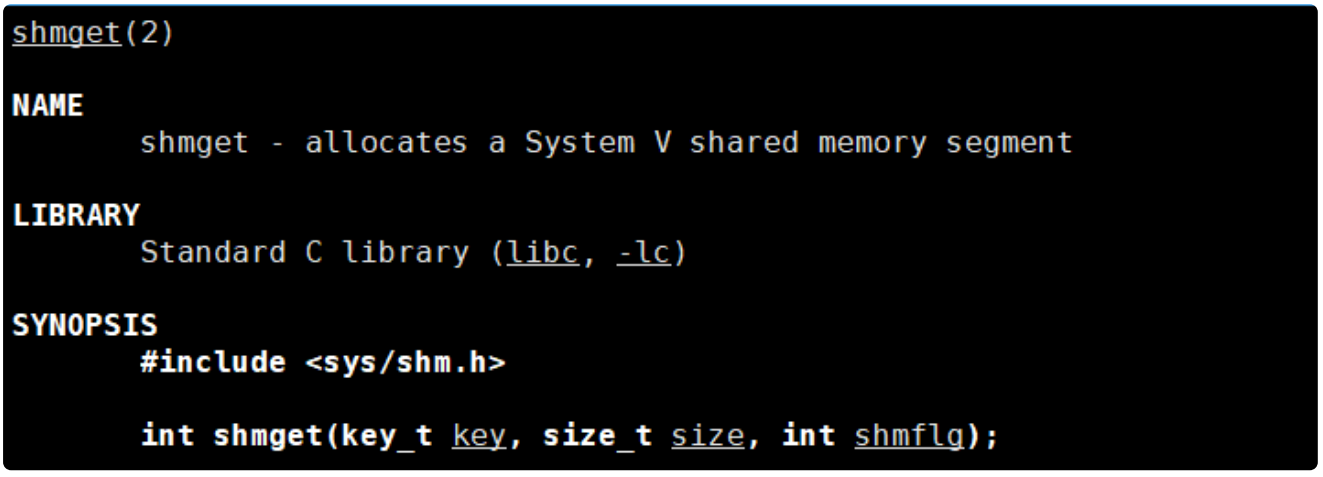
必须区分两个标识：

- `key`：用于让不同进程找到同一个内核 IPC 资源，是双方在程序层面约定的标识。
- `shmid`：`shmget()` 返回给当前程序的共享内存 ID，后续 `shmat()`、`shmdt()`、`shmctl()` 主要使用它。

`key` 不需要每次由内核随机生成，因为内核无法预先知道哪些进程准备通信；双方在程序运行前约定路径和项目编号更直接。

2.4 共享内存核心接口

2.4.1 `shmget`：创建或获取共享内存



```
shmget(2)

NAME
       shmget - allocates a System V shared memory segment

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/shm.h>

       int shmget(key_t key, size_t size, int shmflg);
```

```c
#include <sys/ipc.h>
#include <sys/shm.h>

int shmget(key_t key, size_t size, int shmflg);
```

- 成功时返回非负的 `shmid`。
- 失败时返回 `-1`，并设置 `errno`。
- `size` 表示请求的大小，内核会按页大小对齐。Linux 常见页大小是 `4096` 字节，因此请求 `4097` 字节时实际通常分配 `8192` 字节。
- 权限位应与创建文件时类似，例如 `0666`。

常用标志：

标志	含义
`IPC_CREAT`	不存在则创建，已存在则获取
`IPC_CREAT | IPC_EXCL`	不存在则创建，已存在则失败，适合“只创建新对象”的一方
`0`	只获取已有对象，不主动创建

服务端通常使用：

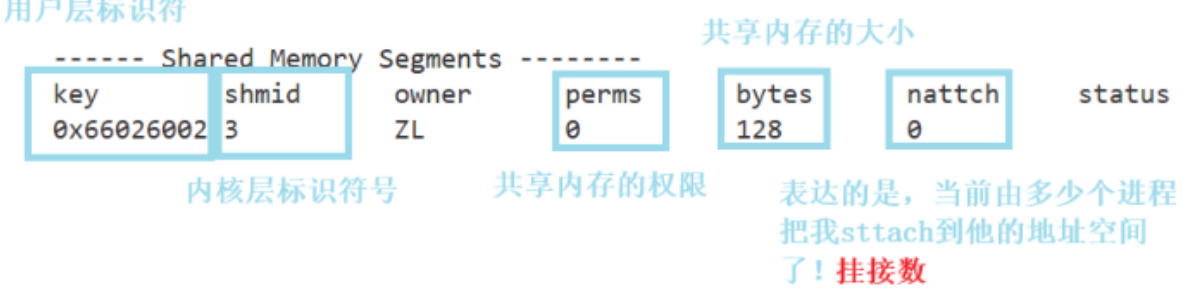
```
int shmid = shmget(key, 4096, IPC_CREAT | IPC_EXCL | 0666);
```

客户端通常使用：

```
int shmid = shmget(key, 4096, 0);
```

若不设置权限位，后续挂接可能因权限不足而失败。

```
ZL@VM-0-14-ubuntu:~/code/lesson24$ ./Server
key = 0x66026002,_shmid = 4
Attach errorZL@VM-0-14-ubuntu:~/code/lesson24$
```



2.4.2 `shmat`：挂接到进程地址空间

```
SHMOP(2)                                                    System Calls Manual

NAME
       shmat, shmdt - System V shared memory operations

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/shm.h>

       void *shmat(int shmid, const void *_Nullable shmaddr, int shmflg);
       int shmdt(const void *shmaddr);
```

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
```

- `shmid`：共享内存 ID。
- `shmaddr`：建议的映射地址，传 `nullptr` 时由内核选择。
- `shmflg`：挂接标志，例如只读标志 `SHM_RDONLY`。

成功时返回映射区域在当前进程中的起始地址；失败时返回 `(void *)-1`，不能只判断是否为 `nullptr`。

```
void *addr = shmat(shmid, nullptr, 0);
if (addr == (void *)-1)
{
    perror("shmat");
    return 1;
}
```

挂接成功后，共享内存的 `nattch` 会反映当前挂接到该段上的进程数量。

```
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status

0x66026002 5          ZL         666        128        1
```

2.4.3 `shmdt`：解除当前进程的挂接

```
SHMOP(2)

NAME
       shmat, shmdt - System V shared memory operations

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/shm.h>

       void *shmat(int shmid, const void *_Nullable shmaddr, int shmflg);
       int shmdt(const void *shmaddr);
```

```
int shmdt(const void *shmaddr);
```

- 参数必须是 `shmat()` 返回的地址。
- 成功返回 `0`，失败返回 `-1`。
- `shmdt()` 只是让当前进程不再映射这段共享内存，不会删除共享内存对象。
- 解除挂接后，共享内存的挂接计数减少。

```
if (shmdt(addr) == -1)
    perror("shmdt");
```

2.4.4 `shmctl`：查询属性与删除资源

```
shmctl(2)                                                System Calls Manual

NAME
       shmctl - System V shared memory control

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/shm.h>

       int shmctl(int shmid, int op, struct shmid_ds *buf);
```

```
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

常用命令：

- `IPC_STAT`：获取共享内存属性。

- `IPC_SET`：修改部分属性。
- `IPC_RMID`：标记删除共享内存。

```
shmctl(shmid, IPC_RMID, nullptr);
```

`IPC_RMID` 是 System V IPC 的关键清理操作。共享内存不会因为创建进程退出而自动消失；如果不显式删除，它会一直保留到系统重启。执行 `IPC_RMID` 后，新进程通常不能再挂接，已在使用的映射会继续有效，直到相关进程解除挂接后由内核回收。

2.5 生命周期、查看与清理

```
ZL@VM-0-14-ubuntu:~/code/lesson24$ ./Server
key = 0x66026002,_shmid = 1
ZL@VM-0-14-ubuntu:~/code/lesson24$ ./Server
shmget: File exists
ZL@VM-0-14-ubuntu:~/code/lesson24$ ./Server
shmget: File exists
ZL@VM-0-14-ubuntu:~/code/lesson24$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x66026002 1          ZL         0          128        0
```

共享内存以及整个 System V IPC 资源的生命周期跟随内核，而不是跟随某一个用户进程。进程退出只会结束它自己的挂接，不会自动删除内核中的共享内存对象。

查看共享内存：

```
ipcs -m
```

持续观察挂接数：

```
while :; do
    ipcs -m
    sleep 1
done
```

命令行删除：

```
ipcrm -m <shmid>
```

注意，`ipcrm -m` 接收的是 `shmid`，不是 `key`。

下面几种状态需要分清：

1. **创建**：使用 `IPC_CREAT` 或 `IPC_CREAT | IPC_EXCL` 建立对象。
2. **挂接**：`shmat()` 后，当前进程能通过虚拟地址访问该段。
3. **解除挂接**：`shmdt()` 取消当前进程的映射。
4. **删除**：`shmctl(..., IPC_RMID, ...)` 或 `ipcrm` 请求内核回收资源。

如果使用 `IPC_CREAT | IPC_EXCL` 再次创建同名资源，可能因为 key 对应的资源已经存在而得到 `EEXIST`。这通常不是 `shmid` 冲突，而是同一 key 已关联共享内存对象。

2.6 容量、权限与越界风险

```
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status

0x66026002 6          ZL         666        4097       0
```

共享内存通常按内存页管理。请求大小不是页大小整数倍时，内核会向上取整到足够的页数。例如请求 `4097` 字节，实际可映射空间通常达到 `8192` 字节。

这不意味着程序可以使用多出来的全部空间。应用仍应把逻辑容量限制在自己约定的范围内，否则越界访问可能不会立刻触发检查，反而破坏相邻内容或让问题难以定位。

共享内存权限通过 `shmget()` 的模式位或后续控制接口设置。通信双方应明确谁创建、谁只读、谁可写，避免把整个区域开放为无约束读写。

2.7 共享内存访问示例

下面用最小封装展示“创建、挂接、访问、解除、删除”的完整顺序：

```cpp
#include <cerrno>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <iostream>
#include <sys/ipc.h>
```

```cpp
#include <sys/shm.h>
#include <unistd.h>

constexpr std::size_t SHM_SIZE = 4096;

class Shm
{
public:
    Shm()
    {
        key_t key = ftok("/tmp", 0x66);
        if (key == -1)
        {
            std::perror("ftok");
            std::exit(1);
        }
        key_ = key;
    }

    bool Create()
    {
        id_ = shmget(key_, SHM_SIZE, IPC_CREAT | IPC_EXCL | 0666);
        return id_ != -1;
    }

    bool Get()
    {
        id_ = shmget(key_, SHM_SIZE, 0);
        return id_ != -1;
    }

    void Attach()
    {
        addr_ = shmat(id_, nullptr, 0);
        if (addr_ == reinterpret_cast<void *>(-1))
        {
            std::perror("shmat");
            std::exit(2);
        }
    }

    void Detach()
    {
        if (addr_ != nullptr && shmdt(addr_) == -1)
            std::perror("shmdt");
        addr_ = nullptr;
```

```
    }

    void Remove()
    {
        if (id_ != -1 && shmctl(id_, IPC_RMID, nullptr) == -1)
            std::perror("shmctl");
    }

    char *Data() { return static_cast<char *>(addr_); }

private:
    key_t key_ = -1;
    int id_ = -1;
    void *addr_ = nullptr;
};
```

创建方：

```
int main()
{
    Shm shm;
    if (!shm.Create())
    {
        std::perror("shmget");
        return 1;
    }

    shm.Attach();
    std::strncpy(shm.Data(), "shared memory ready", SHM_SIZE - 1);

    sleep(10); // 等待另一方读取，仅用于演示
    shm.Detach();
    shm.Remove();
    return 0;
}
```

访问方：

```
int main()
{
    Shm shm;
    if (!shm.Get())
    {
```

```
        std::perror("shmget");
        return 1;
    }

    shm.Attach();
    std::cout << "received: " << shm.Data() << '\n';
    shm.Detach();
    return 0;
}
```

这段代码只演示映射和资源生命周期，没有解决读写时序。生产代码若要求“写完后通知、读取时不会半写”，必须额外使用信号量、互斥量或其他同步机制。

2.8 共享内存为什么必须考虑同步

共享内存只负责建立同一份物理内存的映射，不负责保护其中的数据：

- 多个进程可能同时写同一区域。
- 写进程可能尚未写完，读进程已经开始读取。
- 没有消息边界，也没有自动通知。
- 共享内存本身没有同步或互斥机制。

可以把数据缓冲区继续放在共享内存中，另用一个轻量通知通道传递“可读”事件。这样既能保留共享内存传输数据的高效率，也能避免读者盲目轮询尚未写完的数据。

3. 对比与结论

3.1 命名管道与共享内存的核心差异

对比项	命名管道（FIFO）	System V 共享内存
共享对象	内核管道缓冲区	同一段物理内存页
访问方式	`read()` / `write()`	直接读写映射后的地址
每次传输是否需要系统调用	需要	建立映射后不需要
速度	较慢	IPC 中通常最快
是否保留消息边界	否，字节流	否，由用户约定数据结构
同步与互斥	内核阻塞和原子写提供基础保障	没有，需用户自行实现
生命周期	FIFO 名字可独立存在，通信对象随打开引用管理	随内核，直到显式删除或重启

对比项	命名管道（FIFO）	System V 共享内存
适用对象	任意能访问同一路径的进程	任意能约定同一 key 的进程

3.2 最终理解

命名管道和共享内存分别代表两种思路：

1. 命名管道通过文件系统的名字，让任意进程找到同一个内核字节流对象。
2. 共享内存通过虚拟地址映射，让多个进程直接看到同一段物理内存。

前者接口简单、同步边界更清楚；后者速度最快，但把数据格式、读写顺序和并发保护都交给用户。两者最终都回到 IPC 的本质：

> **进程彼此独立，通信的前提是让它们看到同一份资源；共享资源一旦出现，就还需要进一步解决并发访问问题。**